

1. Definirati: a) striktnu konzistenciju; b) sekvencijalnu konzistenciju; c) uslovnu (kauzalnu) konzistenciju; d) FIFO konzistenciju.

a) Striktna — najjači model, bazira se na postojanju globalnog časovnika. Skladište podataka je striktno konzistentno ako bilo koja operacija nad podatkom X vraća rezultat poslednje write operacije nad tim podatkom X. Nije je moguće postići u DS-u.

b) Sekvencijalna — najjači model koji se može postići u DS-u. Rezultat bilo kog izvršenja je isti kao da su read i write operacije svih procesa na skladištu podataka izvršene u nekom sekvencijalnom redosledu, i operacije svakog pojedinačnog procesa pojavljuju se u toj sekvenci u redosledu koji je određen njihovim programom. Svi procesi vide isti redosled.

c) Uslovna (kauzalna) — slabija od sekvencijalne. Upisi koji su potencijalno uslovljeni moraju da se vide u svim procesima u istom redosledu. Konkurentni upisi se mogu videti u različitom redosledu u različitim procesima. Implementira se pomoću vektorskih časovnika.

d) FIFO — slabija od kauzalne. Upisi koje obavi jedan proces vide se od strane drugih procesa po redosledu po kome su izdati. Upisi različitih procesa mogu se videti u različitom redosledu u različitim procesima.

Hijerarhija: striktna $\square$ sekvencijalna $\square$ kauzalna $\square$ FIFO.

3. Šta može biti argument serverske procedure kod Sun RPC, a šta povratna vrednost u slučaju SunRPC? Kako mora biti deklarisan rezultat serverske procedure i zašto?

Sun RPC poziv udaljene procedure može imati samo jedan ulazni argument i jedan izlazni parametar. Ako treba preneti više parametara ili rezultata, to se postiže preko struktura.

Procedure na serveru uzimaju kao parametar pokazivač na podatak koji se prenosi i vraćaju pokazivač na rezultat.

Rezultat mora biti deklarisan kao static — procedura vraća pokazivač na rezultat, pa promenljiva mora da postoji i nakon povratka iz procedure (lokalna bi bila uništena po izlasku iz funkcije).

```
int *saber1_svc(operandi *a, struct svc_req *rqstp) {
    static int zbir;
    zbir = a->x + a->y;
    return &zbir;
}
```

2. Izlaz iz Sun IDL kompajlera sastoji se od više fajlova. Koji su to fajlovi i šta sadrže? Napisati na koji način se vrši generisanje ovih fajlova i kako se na osnovu generisanih fajlova formiraju izvršna klijentska i serverska aplikacija. Za svaki korak napisati odgovarajuću komandu.

Poziv kompajlera: `rpcgen -C primer.x` — generišu se 3 fajla:

`primer.h` (header) — jedinstveni identifikator interfejsa, definicije tipova, konstanti i prototipova funkcija; uključuje se sa `#include` i u klijent i u server kod.

`primer_clnt.c` (klijent stub) — procedure koje klijent program poziva; pakuju parametre u poruke, šalju poruku, primaju poruku, izvlače rezultat i prosleđuju ga klijentu.

`primer_svc.c` (server stub) — procedure koje se pozivaju kada poruka stigne do servera i koje zatim pozivaju odgovarajuću serversku proceduru.

Koraci i komande:

```
rpcgen -C primer.x
rpcgen -C -Ss primer.x > server.c (template server koda)
cc -o client client.c primer_clnt.c
cc -o server server.c primer_svc.c
```

Pišu se još klijent program (`client.c` — veza preko `clnt_create()` na osnovu imena servera, broja programa i broja verzije) i server program (`server.c` — implementacija procedura, ime oblika `imeprocedure_brojverzije_svc`).

4. Objasniti aktivnosti koje obavljaju klijent stub i server stub prilikom poziva udaljene procedure.

Klijent stub — funkcija koja liči na lokalnu funkciju, ali ne sadrži kod procedure, već kod za slanje parametara i prijem rezultata kroz mrežu:

- pakuje parametre u mrežnu poruku (marshaling), uz konverziju iz lokalnog u standardni format podataka;
- poziva lokalni OS (`SEND`, pa `RECEIVE`) i čeka odgovor od servera;
- raspakuje rezultat iz poruke i vraća ga klijentskom procesu.

Server stub:

- prima poruku od lokalnog OS-a (`RECEIVE`), raspakuje je (unmarshaling) i izvlači argumente;
- poziva željenu serversku proceduru i predaje joj parametre (stavlja ih na stek);
- pakuje rezultat u poruku i poziva lokalni OS (`SEND`).

5. Koji su načini prenosa parametara kod poziva udaljene procedure? Objasniti.

call-by-value (po vrednosti) — vrednosti parametara se kopiraju u stek; ako pozvana procedura modifikuje podatke, originalni podaci se ne modifikuju.

call-by-reference (po referenci) — adrese parametara se smeštaju u stek; ako pozvana metoda modifikuje podatke, oni se zaista u originalu promene.

call-by-copy/restore — vrednosti parametara se stavljaju na stek kao kod poziva po vrednosti, a nakon završetka poziva procedure modifikovane vrednosti se upisuju preko originalnih vrednosti, kao kod poziva po referenci.

Kod RPC-a se koristi copy/restore način prenosa parametara.

6. Kako se prenose skalarni parametri kod RPC? Šta predstavlja standardni format podataka i zašto je neophodan? Navesti jedan njegov primer. Ko obavlja konverziju podataka kod RPC?

Skalarni parametri se prenose po vrednosti.

U DS-u postoji više tipova mašina — svaka može imati različit način predstavljanja brojeva (big-endian, little-endian, dvojični ili jedinični komplement) i karaktera (ASCII, EBCDIC), pa nije moguće preneti parametre onako kako su zapisani lokalno. Zato se koristi „standardno“ kodiranje svih tipova podataka koji se mogu prenositi kao parametri.

Primer: XDR (eXternal Data Representation) kod Sun RPC-a.

Konverziju iz lokalnog u standardni način predstavljanja podataka (i obrnuto) obavljaju klijentski i serverski stub.

9. Šta se dešava kada server promeni mrežnu adresu u sistemu sa dinamičkim povezivanjem?

- ☐ a. Klijent mora ponovo da se kompajlira
- ☒ b. Server se ponovo registruje kod bindera sa novom adresom (TAČNO)
- ☐ c. Binder obaveštava sve aktivne klijente o promeni
- ☒ d. Klijent može da dobije novu adresu pri sledećem pozivu (TAČNO)

7. Navesti gde se deklariraju promenljive čiji su tipovi definisani u XDR-u. Kako se definiše niz promenljive veličine? Kako se predstavljaju nizovi promenljive veličine u generisanom C kodu?

RPC definicije tipova se kompajliraju i nalaze u izlaznom header fajlu (primer.h). Definicijom se ne alokira memorijski prostor — promenljive se naknadno deklariraju u klijent/server kodu. Deklaracije ne mogu stajati samostalno, već moraju biti deo strukture ili u okviru typedef.

Niz promenljive veličine — uglavne zagrade:

```
int heights<50>; /* najviše 50 vrednosti */  
long x_vals<>; /* najviše 2^32-1 vrednosti */
```

U generisanom C kodu predstavlja se strukturom sa dva polja:

```
typedef int heights<50>; -->  
struct { int heights_len; int *heights_val; } heights;
```

gde je heights_len broj elemenata niza, a heights_val pokazivač na prvi element niza.

8. Kako se može locirati server koji implementira udaljenu proceduru?

Postoje 2 tipa povezivanja:

1) Statičko povezivanje — klijent zna koji server treba da kontaktira i adresa tog servera je u klijent stub-u. Kada klijent pozove proceduru, klijent stub samo prosledi poziv serveru čiju adresu ima. Poseban program PORTMAPPER na udaljenom serveru pamti preslikavanja imena programa, broja verzije i broja porta (port 111).

2) Dinamičko povezivanje — postoji centralizovana baza podataka smeštena u Name i Directory serverima koja može locirati server koji obezbeđuje željeni servis. Kada klijent pozove udaljenu proceduru, klijent stub kontaktira Name server da bi dobio adresu servera, pa uspostavlja vezu sa njim. Serverski stub registruje serversku proceduru u Name i Directory serveru prilikom startovanja servera. Ako server promeni adresu, dovoljno je promeniti ulaz u Name serveru.

17. Koji od sledećih zahteva ne mora uvek da važi u message queuing sistemima?

- ☐ a. Pouzdana isporuka poruka
- ☐ b. Redosledna isporuka poruka
- ☒ c. Direktna veza između pošiljaoca i primaoca (TAČNO)
- ☐ d. Tranzijentno skladištenje poruka

10. U distribuiranom sistemu klijent koristi RPC za komunikaciju sa serverom. Klijent šalje zahtev serveru (npr. operation x()) sve dok ne dobije odgovor od servera. a) Koja RPC semantika u slučaju greške je u ovom slučaju implementirana? b) Objasniti kako mora da bude implementiran server da bi se ovakva semantika obezbedila? Navesti bar dve mogućnosti.

a) At-least-once („bar jednom“) semantika — operacija će se izvršiti najmanje jednom, a možda i više puta.

b) Dve mogućnosti na strani servera:

1. Da procedura operation x() bude idempotentna — vraća uvek isti rezultat ma koliko se puta izvrši i ne menja stanje sistema.

2. Da server bude statefull — vodi računa o klijentskim zahtevima, prepoznaje duplikate zahteva i u tom slučaju vrši samo retransmisiju poruke u kojoj je odgovor (ne izvršava ponovo proceduru).

11. Koja semantika poziva udaljenih procedura je podržana kod DCE RPC i kako?

DCE podržava dve semantičke opcije kod poziva udaljene procedure:

„bar jednom“ (at-least-once) — ako su u pitanju idempotentne funkcije (mogu se izvršavati više puta bez posledica). Takve procedure se moraju označiti kao idempotent u IDL-u.

„samo jednom“ (at-most-once) — funkcija nije idempotentna (poziv generiše efekte, npr. modifikovanje podataka). Ovo je podrazumevana semantika.

14. Prilikom uspostavljanja DCE RPC komunikacije, koje tvrdnje su tačne?

✓ A. U DCE RPC sistemu klijent može koristiti CDS/Directory Server da pronađe odgovarajući server koji implementira traženi interfejs. (TAČNO)

✓ B. Nakon pronalaženja servisa, komunikaciju između klijenta i servera dalje obavljaju RPC stubovi i runtime biblioteka. (TAČNO)

✗ C. CDS i rpcbind čuvaju kôd udaljenih procedura koje treba izvršiti.

✗ D. Ako klijent ne pronađe servis u CDS-u ili rpcbind-u, RPC poziv se ipak izvršava jer klijentski stub sadrži kôd udaljene procedure.

✓ E. CDS koristi UUID interfejsa, dok Sun RPC koristi broj programa i verziju za identifikaciju servisa. (TAČNO)

12. Koji tip povezivanja između klijenta i servera je podržan u DCE RPC-u? Opišite postupak povezivanja.

Podržano je dinamičko povezivanje — aplikacije identifikuju resurse po imenu, bez potrebe da znaju gde je resurs lociran. Koristi se CDS (Cell Directory Service) koji preslikava logičko ime u IP adresu servera unutar DCE ćelije; imenima van lokalne ćelije upravlja GDS.

DCE klijent pronalazi server u 2 koraka: lociranje serverske mašine, pa lociranje odgovarajućeg procesa na njoj. Postupak povezivanja:

1. Registrovanje broja porta servera (procesa) u DCE deamonu (održava tabelu parova [server, brojPorta]);
2. Registrovanje servera u Directory serveru (adresa serverske mašine, ime servera i interfejsi koje implementira);
3. Klijent kontaktira Directory server da dobije IP adresu server mašine na kojoj se izvršava željeni server proces;
4. Klijent kontaktira DCE deamon da bi dobio broj porta server procesa kome želi da pristupi;
5. Poziv udaljene procedure.

13. Šta se dobija kao rezultat poziva uuidgen programa kod DCE RPC? Koji su naredni koraci u pisanju DCE klijent/server aplikacije?

uuidgen imefajla.idl kreira fajl imefajla.idl koji sadrži globalno jedinstveni identifikator interfejsa (UUID) i verziju i garantuje da se isti id neće nigde pojaviti u DS-u:

```
[ uuid(3d6ead56-06e3-11ca-8dd1-826901beabcd), version(1.0) ]  
interface INTERFACENAME { }
```

Naredni koraci:

1. Editovanje IDL fajla — upisuju se ime interfejsa i deklaracije (potpisi) udaljenih procedura;
2. Poziv IDL kompajlera: idl primer.idl — izlaz su 3 fajla: header (primer.h), klijent stub (primer_cstub.c) i server stub (primer_sstub.c);
3. Pisanje klijent i server programa, kompajliranje i povezivanje (linkovanje) sa odgovarajućim stub-om.

DCE RPC programer u opštem slučaju piše 3 programa: klijent kod, server inicijalizacijski kod (registracija u Directory servisu) i server operativni kod.

15. Interfejs koji je na raspolaganju aplikacijama za razmenu poruka kod Message queuing sistema. Objasniti.

Osnovni interfejs je veoma jednostavan i sadrži 4 operacije: put, get, poll i notify.

put — poziva je pošiljalac; prosleđuje poruku sistemu koja treba biti dodata u specificirani red. Neblokirajući poziv.

get — blokirajući poziv koji omogućava autorizovanom procesu da iz specificiranog reda pribavi poruku koja je najduže u njemu. Proces se blokira ako je red prazan. Varijacije omogućavaju traženje specifične poruke po sadržaju.

poll — neblokirajuća varijanta get operacije. Ako je red prazan ili specifična poruka ne postoji, pozivajući proces nastavlja sa izvršenjem.

notify — omogućava procesu da instalira handler koji se automatski poziva kad god je poruka dodata u red. Često implementirano kroz demon na strani primaoca koji osluškuje dolazeće poruke.

16. Interfejs koji je na raspolaganju aplikacijama za razmenu poruka kod publish-subscribe sistema.

Umesto pošiljaoca i primaoca postoje Publisher (izdavač) i Subscriber (pretplatnik); umesto reda, destinacija je Topic (tema). Poruku može primiti više Subscriber-a.

publish(event) — izdavač distribuira događaj u sistem; događaj se emituje svim pretplatnicima koji su izrazili interesovanje. Izdavači ne moraju znati identitete pojedinačnih pretplatnika.

subscribe(filter) — pretplatnik izražava interesovanje za primanje određenih tipova događaja; filter definiše kriterijume (po tipu događaja, atributima itd.).

notify(event) — sistem se pobrine da svi pretplatnici čiji filteri odgovaraju događaju dobiju obaveštenja i isporučuje im događaj.

unsubscribe(filter) — pretplatnik povlači interesovanje; sistem uklanja pretplatu.

Filtriranje može biti po temi (pretplata na teme) ili po sadržaju (kriterijumi nad sadržajem događaja).

18. Koje od navedenih tvrdnji tačno opisuje karakteristike i rad sistema zasnovanih na redovima poruka (Message Queues)?

- ✓ A. Poruka se šalje u određeni red koji predstavlja odredište. (TAČNO)
- ✓ B. Više primalaca može čitati poruke iz istog reda u zavisnosti od modela rada. (TAČNO)
- ✗ C. Pošiljalac mora znati IP adresu svakog primaoca.
- ✓ D. Red poruka odvaja pošiljaoca od konkretne implementacije primaoca. (TAČNO)
- ✗ E. Promena primaoca zahteva promenu svih pošiljalaca.

19. Šta predstavlja JMS administrativni objekat, koja je njegova uloga i koji su primeri takvih objekata? Iz čega se sastoji JMS poruka i kako i na osnovu čega se može vršiti filtriranje poruke u JMS?

Administrativni objekti u JMS su unapred konfigurisani JMS objekti koje kreira administrator za upotrebu od strane klijenata. Služe kao „most“ između kôda klijenta i JMS provajdera. Primeri: fabrike konekcija (omogućavaju klijentima da uspostave konekcije sa JMS provajderom) i destinacije (JMS tema ili JMS red).

JMS poruka se sastoji od tri dela:

1. Zaglavlje — informacije za identifikaciju i rutiranje poruke: odredište, prioritet, datum isteka, ID poruke i vremenska oznaka;
2. Svojstva (properties) — definisana od strane korisnika, povezuju dodatne metapodatke aplikacije sa porukom;
3. Telo — tekstualna poruka, niz bajtova, serijalizovani Java objekat, niz primitivnih Java vrednosti ili skup parova ime/vrednost.

Filtriranje se vrši selektorom poruke — predikatom definisanim preko vrednosti u delovima zaglavlja i svojstava poruke (ne tela).

24. Koji sistem je otporan na greške Vizantijskog tipa?

- ✗ a. Sistem koji je otporan na otkaz više čvorova
- ✗ b. Sistem koji je otporan na gubitak više poruka
- ✗ c. Sistem koji je otporan na podelu mreže
- ✓ d. Sistem koji je otporan na maliciozne čvorove (TAČNO)

20. Šta predstavlja konekcija a šta sesija kod JMS? Koji tipovi konekcija su podržani kod JMS? Koji deo JMS poruke može biti korišćen prilikom definisanja selektora poruka?

Konekcija — uspostavlja se kroz fabriku konekcija; rezultujuća konekcija je logički kanal između klijenta i JMS provajdera.

Sesija — serija operacija koje uključuju kreiranje, proizvodnju i konzumaciju poruka vezanih za logički zadatak. Objekat sesije je centralan za rad JMS-a i podržava i operacije za kreiranje transakcija (sve-ili-ništa).

Tipovi konekcija: TopicConnection i QueueConnection.

- ☐ A. Samo telo poruke
- ☐ B. Samo zaglavlje poruke
- ☒ C. Zaglavlje i svojstva poruke, ali ne i telo poruke (TAČNO)
- ☐ D. Zaglavlje, svojstva i telo poruke

21. Šta sadrži zaglavlje JMS poruke? Koja je uloga svojstava JMS poruke? Navesti i objasniti načine prijema poruka u JMS-u i navesti mehanizme koji se koriste za realizaciju svakog načina prijema.

Zaglavlje sadrži sve informacije potrebne za identifikaciju i rutiranje poruke: odredište (referenca na temu ili red), prioritet poruke, datum isteka, ID poruke i vremensku oznaku. Većinu polja kreira JMS provajder, a neka može popuniti korisnik.

Svojstva (properties) su definisana od strane korisnika i koriste se za povezivanje dodatnih metapodataka aplikacije sa porukom (npr. polje za lokaciju).

Dva načina prijema poruka:

1. Sinhroni (blokirajući) — program blokira korišćenjem operacije primanja (receive) i čeka dok poruka ne stigne;
2. Asinhroni — program uspostavlja objekat slušaoca poruka (MessageListener) koji mora obezbediti metodu onMessage, koja se poziva kad god se identifikuje odgovarajuća dolazeća poruka.

36. Šta je definicija potpuno uređene grupne komunikacije?

- ☐ a) poruke se isporučuju procesima u FIFO redosledu
- ☐ b) poruke se procesima isporučuju po redosledu realnog vremena
- ☐ c) poruke se isporučuju procesima po redosledu „desilo se pre"
- ☒ d) poruke se isporučuju svim procesima po istom redosledu (TAČNO)

22. Koji tipovi grešaka u odnosu na trajanje postoje? Dati primer za svaku od njih. Objasniti razliku između „mirnih“ i Vizantijskih grešaka.

Prolazne (transient) — pojave se jednom i nestanu. Primer: istekao timeout za prenos poruke, ali nakon ponovnog slanja poruka je uspešno otposlata.

Periodične (intermittent) — greška se pojavi, nestane, pa se ponovo pojavi. Najnezgodniji tip grešaka. Primer: greška u mrežnom kablju kod koga postoji povremeni prekid.

Stalne (permanent) — postoje sve dok se komponenta ne zameni ispravnom. Primer: greška u hard disku, pregorela komponenta, bagovit softver.

Mirna greška — komponenta prestaje sa radom i ne generiše nikakav rezultat ili generiše poruku o grešci.

Vizantijska greška — komponenta nastavlja sa radom i generiše pogrešan rezultat; nema jasne naznake da li je komponenta otkazala.

23. Šta se podrazumeva pod raspoloživošću a šta pod pouzdanošću sistema? Koliko je replika potrebno ako se mora tolerisati: a) f mirnih grešaka; b) f grešaka Vizantijskog tipa; c) f grešaka Vizantijskog tipa pri čemu sistem mora postići konsenzus?

Raspoloživost — sistem koji je spreman za korišćenje u trenutku kada je to potrebno.

Pouzdanost — osobina sistema da kontinualno radi (duže vreme) bez nastupanja grešaka.

Razlika je u vremenu: raspoloživost se odnosi na konkretan trenutak, a pouzdanost na dug vremenski period.

a) f mirnih grešaka → $f + 1$ replika

b) f Vizantijskih grešaka → $2f + 1$ replika (većinsko glasanje)

c) f Vizantijskih grešaka uz konsenzus → $3f + 1$ replika (bar $2f+1$ lojalnih, više od $2/3$)

27. Šta se podrazumeva pod transparentnošću konkurencije?

X a) sve niti mogu pristupati deljivim strukturama podataka

✓ b) procesi mogu pristupati resursima bez međusobne interferencije (TAČNO)

X c) repliciranim resursima se pristupa kao da postoji samo jedna kopija

X d) novi čvorovi se mogu dodati sistemu bez promene aplikacije

25. Klijent je uputio zahtev serveru. Odgovor nije stigao. Da li klijent može da zaključi da li je došlo do otkaza servera ili je izgubljen odgovor? Obrazložiti.

Ne može. Klijent ne može da razlikuje ove situacije, jer on vidi samo istek timeout-a — otkaz servera i izgubljen odgovor izgledaju identično sa strane klijenta.

Zato se ne može garantovati exactly-once semantika, već klijent bira između at-least-once (ponavlja zahtev — samo za idempotentne procedure) i at-most-once (odmah signalizira grešku).

26. Šta predstavlja skalabilnost distribuiranog sistema? Navesti i objasniti tehnike skaliranja distribuiranih sistema.

Skalabilnost (proširljivost) — mogućnost proširenja sistema, tj. dodavanja novih računara. Posmatra se kroz 3 dimenzije: skalabilnost u odnosu na broj korisnika i resursa; u odnosu na geografsku udaljenost resursa i korisnika; administrativna skalabilnost (sistemom se može lako upravljati i kroz više administrativnih domena).

Tehnike skaliranja (3):

1. Skrivanje komunikacionog kašnjenja — koriste se asinhronne komunikacije umesto sinhronih; klijent se ne blokira dok čeka odgovor, već radi drugi posao, a prekid ga obaveštava kada odgovor stigne. Kod interaktivnih aplikacija rešenje je download-ovati deo koda na klijentsku stranu.

2. Distribucija — komponente se dele na manje delove, a zatim se ti delovi distribuiraju na više mašina u sistemu. Primer: DNS i Web.

3. Replikacija — kopija resursa se postavlja blizu mesta korišćenja da bi se smanjilo komunikaciono kašnjenje i postigle bolje performanse.

Problem tehnika skaliranja: više kopija dovodi do problema konzistencije — za usklađivanje kopija bila bi potrebna globalna sinhronizacija, koju je gotovo nemoguće postići u DS-u.

29. Šta se podrazumeva pod transparentnošću replikacije?

☐ a) proces je svestan šeme replikacije i može to iskoristiti

☒ b) repliciranim resursima se pristupa kao da postoji samo jedna kopija (TAČNO)

☐ c) resurs će upravljati svim zahtevima na isti način bez obzira na lokaciju klijenta

☐ d) podaci se ne mogu menjati i mogu se zapamtiti na disku

28. Koje su osobine sistema koji poseduje transparentnost konkurencije?

- ✓ a) Rezultat istovremenih zahteva isti je kao i kada bi se izvršavali sekvencijalno. (TAČNO)
- ✓ b) Korisnik ne mora da zna da postoje drugi konkurentni korisnici. (TAČNO)
- ✗ c) Korisnik mora ručno sinhronizovati pristup deljenim resursima
- ✓ d) Sistem sprečava nekonzistentnost podataka tokom istovremenog pristupa. (TAČNO)

30. Šta se podrazumeva pod pristupnom transparentnošću?

- ✗ A. udaljenim resursima se pristupa korišćenjem lokaciono nezavisnih imena
- ✓ B. lokalnim i udaljenim resursima se pristupa korišćenjem istih operacija (TAČNO)
- ✗ C. repliciranim resursima se pristupa kao da postoji samo jedna kopija
- ✗ D. resurs će upravljati svim zahtevima na isti način bez obzira na lokaciju klijenta

31. Objasniti migracionu transparentnost i transparentnost paralelizacije.

Migraciona transparentnost — resurs može promeniti svoju lokaciju a da klijent to ne primeti ni zna. Resurs se može kretati a da pri tome ne dolazi do promene imena resursa. Primer: mobilni telefoni — pozivalac i pozvani nisu svesni kretanja onog drugog.

Transparentnost paralelizacije — paralelizacija procesa se izvršava transparentno za aplikativnog programera i korisnika aplikacije. Korisnik nije svestan da i kako je aplikacija paralelizovana i gde se izvršavaju procesi. Primer: Hadoop.

34. Ako se koriste vektorski časovnici, šta je najviše što možemo doznati?

- ✗ a) ako je $V(a) < V(b)$ tada se a desilo pre b
- ✓ b) $V(a) < V(b)$ ako i samo ako se a desilo pre b (TAČNO)
- ✗ c) ako se a desilo pre b tada je $V(a) < V(b)$
- ✗ d) ako je $V(a) = V(b)$ tada su a i b neuredjeni

32. Kroz koje sve aspekte se manifestuje heterogenost u distribuiranim sistemima?

DS je sastavljen od heterogenog skupa računara. Heterogenost se ogleda u sledećem:

1. Hardver računara — različit skup instrukcija, različita interna prezentacija podataka;
2. Operativni sistemi — interfejs za razmenu poruka se razlikuje od OS do OS (npr. Berkeley i WinSock);
3. Programski jezici — karakteri i strukture podataka se različito predstavljaju u različitim programskim jezicima, što može biti problem ako aplikacije treba međusobno da komuniciraju;
4. Implementacije od strane različitih projekatata — dok se zajednički standardi ne usvoje, različite implementacije ne mogu međusobno da komuniciraju (OSI model je jedan način za rešavanje problema heterogenosti).

33. Šta je funkcija middleware u distribuiranom sistemu?

Osnovni cilj middleware-a je da se sakrije heterogenost platforme na kojoj je sistem izgrađen od same aplikacije, kao i da se sakrije komunikacija. Mrežni OS se nadograđuje dodatnim softverskim slojem — middleware-om.

Middleware je računarski softver koji pruža usluge (servise) višeg nivoa softverskim aplikacijama (npr. autentifikacija i autorizacija). Middleware komunikacioni protokoli oslobađaju aplikativnog programera detalja vezanih za komunikaciju između procesa — komunikacija je skrivena iza poziva procedure (RPC) ili metoda (RMI).

35. Šta će se dogoditi ako se koristi Ricart-Agrawala algoritam i dva procesa koji žele pristup istoj kritičnoj sekciji generišu zahtev sa istom vrednošću Lamportove markice?

Moguće je da više konkurentnih događaja ima iste vremenske markice, što nije poželjno — to dovodi do konfuzije ako više procesa treba da donese odluku na osnovu vremenskih markica dva događaja (pobednik se bira po manjoj markici, pa se remi ne može razrešiti).

Rešenje: prisiliti da svaka markica bude jedinstvena — Lamportovoj vremenskoj markici dodati još jedan identifikator koji predstavlja globalno jedinstveni id procesa u kome je nastao događaj (adresa hosta + proces ID). Markica postaje par (T, id) i poredi se prvo po T , a kod jednakog T pobeđuje manji id procesa.

37. Šta je definicija potpuno uređene grupne komunikacije? (otvoreno pitanje)

Potpuno uređena grupna komunikacija odnosi se na komunikaciju u kojoj sve replike treba da prime isti skup poruka u istom redosledu, tj. da sve replike budu u konzistentnom stanju.

Obezbediti potpuno uređenu grupnu komunikaciju znači obezbediti operaciju (multicast) kojom se sve poruke isporučuju u istom redosledu svim prijemnicima.

38. Koje časovnike ćete koristiti ako je potrebno ostvariti: a) totalno uređenu grupnu komunikaciju (multicast); b) uslovno (kauzalno) uređenu grupnu komunikaciju?

a) Totalno uređena grupna komunikacija → Lamportovi (skalarni) logički časovnici. Poruka ažuriranja se obeležava logičkim vremenom izvora (proširena markica T.id), poruke se u redovima čekanja uređuju po markicama i isporučuju aplikaciji tek kada su potvrđene (ACK) od strane svih procesa — tako svi procesi vide isti redosled.

b) Uslovno (kauzalno) uređena grupna komunikacija → vektorski časovnici. Časovnik se inkrementira samo kod slanja poruke. Poruka m se ne prosleđuje procesu Pj dok se ne zadovolje dva uslova: $tm[i] = Vj[i] + 1$ (Pj je primio sve prethodne poruke od Pi) i $tm[k] \leq Vj[k]$ za svako $k \neq i$ (Pj je primio sve poruke koje je Pi primio pre slanja m); u suprotnom se poruka baferuje.

39. Objasniti razliku između potpuno uređene grupne komunikacije (total ordering) i uređenja međusobno zavisnih događaja (causal ordering). Da li totalno uređenje garantuje kauzalno uređenje?

Totalno uređenje — sve poruke se isporučuju svim procesima u istom redosledu (i konkurentne). Implementira se Lamportovim časovnicima.

Kauzalno uređenje — samo potencijalno uslovljene (kauzalno povezane) poruke moraju se u svim procesima videti u istom redosledu; konkurentne poruke se mogu videti u različitom redosledu u različitim procesima.

Implementira se vektorskim časovnicima.

Da li totalno garantuje kauzalno? NE. Totalno uređenje garantuje samo da svi procesi vide isti redosled, ali ne i da je taj redosled saglasan sa kauzalnošću — moguće je da svi procesi isporuče odgovor m2 pre poruke m1 koja ga je izazvala. Zato kauzalno uređenje zahteva posebnu implementaciju vektorskim časovnicima sa baferovanjem.

40. Koje vrste replika postoje? Koje se informacije mogu proslediti replikama kada se obavlja ažuriranje? Koji protokol konzistencije se koristi za postizanje sekvencijalne konzistencije?

Vrste replika (3):

1. Permanentne replike — početni skup replika koje obrazuju distribuirano skladište podataka; broj im je relativno mali.
2. Replike inicirane od strane servera — privremene, postavljaju se u regione odakle dolazi veliki broj zahteva; ako broj obraćanja fajlu pređe prag replikacije, kreira se replika bliže klijentima.
3. Replike inicirane od strane klijenta — keš na klijentu ili proxy serveru; server nema nikakvu odgovornost da održava ove kopije konzistentnim, to radi klijent.

Šta se prosleđuje pri ažuriranju (3 opcije): 1) samo obaveštenje o obavljenom ažuriranju (invalidacija) — pogodno kad je odnos upisa naspram čitanja veliki; 2) modifikovani podaci — pogodno kad je broj čitanja mnogo veći od broja modifikacija; 3) operacije koje su izazvale ažuriranje (aktivna replikacija) — minimalan saobraćaj, ali više procesorskog vremena.

Protokol za sekvencijalnu konzistenciju: protokoli zasnovani na postojanju primarne kopije — remote-write (primar fiksiran na jednoj lokaciji; primar uređuje sve zahteve na globalno jedinstven način pa svi procesi vide sve write operacije u istom redosledu) i local-write (primar privremeno migrira na lokalnu kopiju).

41. Bankovni server je repliciran. Razmatraju se dva dizajna: i) sve replike izvršavaju sve zahteve; ii) jedan server izvršava zahteve i periodično ažurira replike. Koje tehnike repliciranja se koriste u slučaju i) a koje u slučaju ii)?

i) **AKTIVNA REPLIKACIJA** — koristi se kod grupe ravnopravnih procesa (ravne grupe). Klijentski zahtev se prosleđuje svim procesima u grupi i svi ga obrađuju u istom redosledu, što zahteva potpuno uređenu grupnu komunikaciju. Klijentski interfejs koristi većinsko glasanje da prosledi tačan odgovor klijentu.

ii) **PASIVNA REPLIKACIJA (primary-backup)** — koristi se kod hijerarhijski organizovanih grupa: primar obavlja ceo posao, ostali su backup. Ako primar otkáže (detektuje se preko heartbeat poruka), jedan od backup servera preuzima posao — bira se novi primar konsenzusom.

42. Da li Ricart-Agrawala algoritam uzajamnog isključivanja koristi logičke časovnike? Kratko obrazložiti odgovor.

DA, koristi — algoritam koristi logičke časovnike i grupnu komunikaciju.

Kada proces želi da uđe u kritičnu sekciju, generiše poruku koja sadrži: ID procesa, ime željenog resursa i logički časovnik (Lamportovu vremensku markicu), i šalje je svim procesima u grupi.

Logički časovnik je neophodan za razrešavanje sukoba: kada dva procesa istovremeno traže istu kritičnu sekciju, svaki poredi markicu svog zahteva sa markicom primljenog — proces sa manjom vremenskom markicom pobeđuje.

43. Zašto je u DS teško postići savršenu sinhronizaciju časovnika?

Osnovni razlog: ne postoji globalni časovnik — svaka mašina ima svoj časovnik, pa procesi na različitim računarima imaju različitu predstavu o vremenu.

- Tajmer je kvarcni kristal čija brzina oscilacije zavisi od vrste kristala, načina sečenja i veličine napona — ako sistem ima N računara, svih N kristala osciluje sa neznatno različitim brzinama, pa softverski časovnici postepeno ispadaju iz sinhronizma;
- sve metode sinhronizacije svode se na razmenu vrednosti časovnika između računara, a tu je problem komunikaciono kašnjenje — dok poruka sa vremenom stigne, to vreme je već zastarelo;
- tokom vremena časovnici opet izađu iz sinhronizacije, pa se sinhronizacija mora stalno ponavljati.

44. Kristijanov algoritam usvaja da je:

- ☐ a) Kašnjenje kroz mrežu tačno poznato
- ☒ b) Kašnjenja zahteva i odgovora su približno jednaka (TAČNO)
- ☐ c) Serverski časovnik je uvek sporiji
- ☐ d) Svi klijenti međusobno komuniciraju da bi odredili tačno vreme

45. Zašto Berkeley algoritam šalje korekciju vremena umesto tačnog vremena?

Berkeley algoritam obezbeđuje internu sinhronizaciju mašina unutar DS-a bez eksternog izvora tačnog vremena: server vremena periodično proziva svaku mašinu, izračunava srednje vreme i saopštava svim mašinama kako da podese svoje časovnike.

1. Vreme nikad ne sme da ide unazad — ako bi se slalo apsolutno vreme, mašini koja žuri časovnik bi skočio unazad. Sa korekcijom mašina zna koliko i u kom smeru da podesi, a negativnu korekciju primenjuje postepeno (usporavanjem časovnika preko holding registra).
2. Eliminise se komunikaciono kašnjenje — apsolutno vreme zastari dok putuje kroz mrežu, a relativna korekcija ostaje ispravna bez obzira na to kada stigne.
3. Cilj nije globalno tačno vreme, već zajedničko vreme — bitno je da računari jednog DS-a imaju istu predstavu o vremenu, ne mora to vreme da bude globalno tačno.

46. Koje informacije o vremenu se nalaze u poruci koja stiže od NTP servera? Kako to klijent koristi da podesi svoj časovnik?

Vremenska polja u NTP poruci:

- Reference timestamp — vreme kada je sistemski časovnik poslednji put bio postavljen/korigovan;
- Origin timestamp (t_0) — kada je poruka upućena od klijenta ka serveru;
- Receive timestamp (t_1) — kada je poruka stigla od klijenta na server;
- Transmit timestamp (t_2) — kada server šalje odgovor klijentu.

Kako klijent podešava časovnik: klijent beleži i t_3 — trenutak prijema odgovora — pa računa:

kružno vreme propagacije: $d = (t_1 - t_0) + (t_3 - t_2)$

offset casovnika: $o = [(t_1 - t_0) + (t_2 - t_3)] / 2$

tačno vreme: $t_3' = t_3 + o$

Ako je dobijeno vreme veće od trenutnog, časovnik se postavlja direktno; ako je manje, korekcija se vrši postepeno (usporavanjem časovnika), jer vreme ne sme da ide unazad.

47. Navesti sve demone u Hadoop klasteru, objasniti njihove uloge kao i gde se izvršavaju u Hadoop klasteru. Šta predstavlja blok i koje su prednosti korišćenja blokova kod HDFS?

Postoje dve vrste demona: HDFS demoni i MapReduce demoni. U HDFS demone spadaju:

NameNode (NN) — samo jedan, izvršava se na glavnom (master) čvoru. Centralni kontroler HDFS-a: održava fajl sistem namespace, čuva metapodatke (kako su fajlovi podeljeni na blokove, koji slave čvorovi čuvaju koje blokove), nadgleda ponašanje DataNode-ova i koordiniše pristup podacima. Ne čuva podatke nad kojima se vrši obrada; vodi računa da svaki blok zadovolji faktor replikacije.

DataNode (DN) — više njih, na slave čvorovima. Odgovoran za čuvanje podataka u blokovima, primanje naredbi od NN i davanje informacija NN-u. Šalje heartbeat svake 3 sekunde; svaki 10. heartbeat je izveštaj o blokovima.

Secondary NameNode — na serveru na kome nije NN; nije backup NN. Radi konkurentno sa NN kao pomoćni demon: u regularnim intervalima preuzima edit log od NN, spaja ga sa starim checkpoint-om i formira novi checkpoint koji vraća NN-u (koristi se pri sledećem restartu).

Blok: Hadoop koristi blokove da sačuva fajl ili delove fajla; Hadoop blok je fajl na fajl sistemu koji se nalazi u osnovi. Podrazumevana veličina je 64MB (novije verzije 128MB).

Prednosti blokova: 1) imaju fiksnu veličinu pa je lako izračunati koliko će ih stati na disk; 2) raspodelom na više čvorova omogućavaju da fajl bude veći nego bilo koji pojedinačni disk u klasteru; 3) blokovi se repliciraju na više čvorova, što omogućava HDFS-u da bude otporan na greške (Fault Tolerant).

48. Sledeće pitanje odnosi se na HDFS. Tačna tvrđenja obeležite sa T(tačno) a netačna sa F(netačno).

- ✓ a. Faktor replikacije može se konfigurisati na nivou klastera i takođe na nivou fajla. — T (TAČNO)
- ✓ b. Izveštaj o blokovima sa svakog DataNode čvora sadrži listu svih blokova koji su smešteni na tom DataNode čvoru. — T (TAČNO)
- ✓ c. Korisnički podaci se skladište na lokalnom fajl sistemu DataNode čvorova. — T (TAČNO)
- ✗ d. DataNode čvor zna kojim fajlovima pripadaju blokovi koji su na njemu smešteni. — F (NETAČNO)

49. Koji fajlovi se trajno pamte na lokalnom disku NameNode-a? Koje informacije se ne pamte trajno na lokalnom disku, a neophodne su prilikom restarta NameNode-a? Odakle se dobijaju te informacije?

Trajno se pamte na lokalnom disku NN-a (2 fajla):

- fsimage (Checkpoint image / Namespace fajl) — snapshot metapodataka fajl sistema;
- edit log — fajl u kome se trajno čuvaju sve promene koje se dešavaju u HDFS-u.

Pri restartu NN promene iz edit loga se primenjuju na poslednju verziju Namespace fajla (fsimage) da bi se dobila nova verzija metapodataka fajl sistema.

Ne pamti se trajno: preslikavanje blokova na DataNode čvorove (na kom DN-u se koji blok fizički nalazi) — NN te podatke drži samo u glavnoj memoriji, jer se u velikim sistemima često menjaju.

Odakle se dobijaju: iz izveštaja o blokovima (block report) koje DataNode-ovi šalju NameNode-u — svaki 10. heartbeat je izveštaj u kome DN prijavljuje koje blokove čuva.

50. Sledeće pitanje odnosi se na HDFS. Tačna tvrđenja obeležite sa T(tačno) a netačna sa F(netačno).

- X a. Svaki fajl je podrazumevano podeljen na 32 MB po defaultu — F (NETAČNO)
- X b. NameNode čuva podatke fajla u obliku blokova podataka — F (NETAČNO)
- X c. HDFS se koristi za scenarije koji zahtevaju istovremeno upisivanje u istu datoteku — F (NETAČNO)
- ✓ d. Faktor replikacije se može konfigurisati na nivou klastera (podrazumevano je podešeno na 3) i takođe na nivou fajla — T (TAČNO)